

LABORATORIO N° 01

Diseño y Desarrollo de Simuladores y Videojuegos



DOCENTE:

Bestard Aroche, Yunion

CURSO:

Desarrollo de Aplicaciones
Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

Capacidad Terminal 1

- Identificar las características de Django como framework de desarrollo web.

Capacidad Terminal 2

- Instalar y configurar el entorno de desarrollo (Python, entorno virtual y Django).

Capacidad Terminal 3

- Crear una aplicación Web en Django, reconociendo su estructura y elementos principales (proyecto, app, modelos, vistas, URLs y administrador).

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

III. Normas empleadas

- *No aplica*

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Python 3.10 o superior, Django 5, un editor de código (Visual Studio Code) y una cuenta de GitHub para el control de versiones.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Procedimiento

Introducción al laboratorio

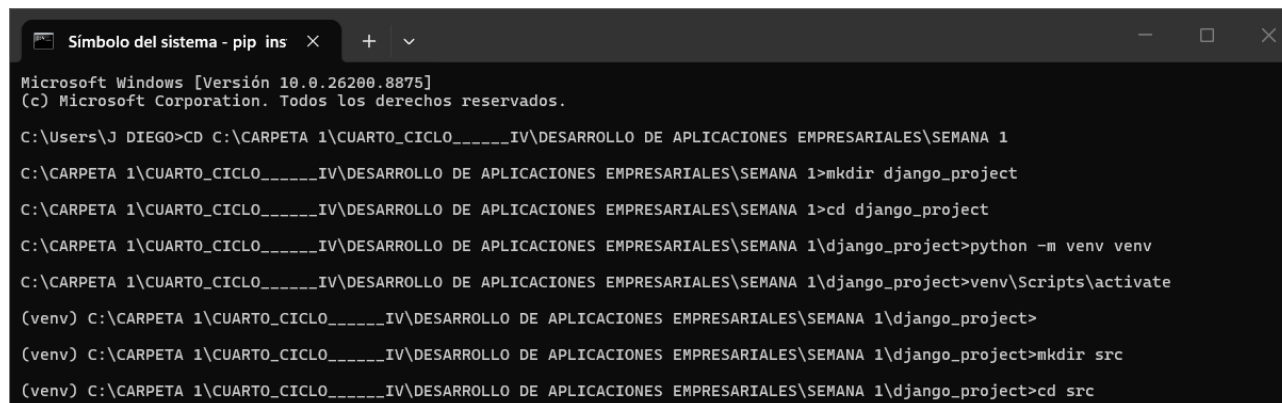
En este laboratorio construirás la primera versión de tu “Sistema Empresarial”: una aplicación Web en Django que gestiona un catálogo simple de ítems (por ejemplo, productos). Este proyecto servirá de base para las siguientes sesiones del curso.

Ejercicios

Desarrolla, en orden, los siguientes ejercicios. No copies comandos sin comprenderlos: cada paso debe quedar evidenciado con una captura de pantalla y una breve explicación.

1. Preparar el entorno de trabajo. Crea la carpeta del proyecto (django_project), crea y activa un entorno virtual de Python (venv), y dentro de él crea la carpeta src/ que contendrá el código fuente.

Figura 1



```

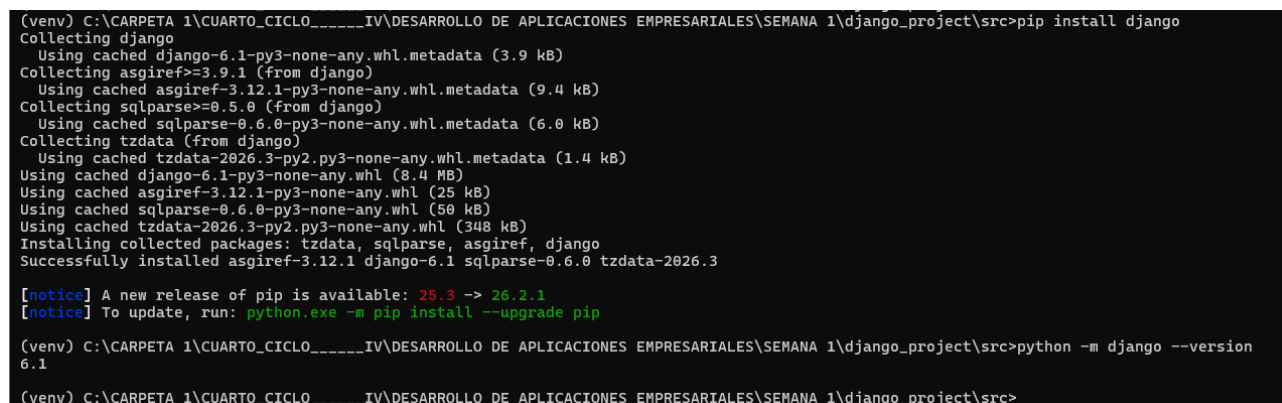
Microsoft Windows [Versión 10.0.26200.8875]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\J DIEGO>CD C:\CARPETA 1\CUARTO_CICLO_____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1
C:\CARPETA 1\CUARTO_CICLO_____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1>mkdir django_project
C:\CARPETA 1\CUARTO_CICLO_____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1>cd django_project
C:\CARPETA 1\CUARTO_CICLO_____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project>python -m venv venv
C:\CARPETA 1\CUARTO_CICLO_____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project>venv\Scripts\activate
(venv) C:\CARPETA 1\CUARTO_CICLO_____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project>
(venv) C:\CARPETA 1\CUARTO_CICLO_____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project>mkdir src
(venv) C:\CARPETA 1\CUARTO_CICLO_____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project>cd src
  
```

Se creó la carpeta del proyecto django_project y dentro un entorno virtual con python -m venv venv, que permite aislar las dependencias de este proyecto sin afectar otros. Se activó con venv\Scripts\activate (se nota por el (venv) al inicio de la línea) y se creó la carpeta src/, que contendrá el código fuente del proyecto.

2. Instalar Django. Dentro del entorno virtual activado, instala Django (versión 5) usando pip y verifica la instalación consultando su versión.

Figura 2



```

(venv) C:\CARPETA 1\CUARTO_CICLO_____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project\src>pip install django
Collecting django
  Using cached django-6.1-py3-none-any.whl.metadata (3.9 kB)
Collecting asgiref<=3.9.1 (from django)
  Using cached asgiref-3.12.1-py3-none-any.whl.metadata (9.4 kB)
Collecting sqlparse<=0.5.0 (from django)
  Using cached sqlparse-0.6.0-py3-none-any.whl.metadata (6.0 kB)
Collecting tzdata (from django)
  Using cached tzdata-2026.3-py2.py3-none-any.whl.metadata (1.4 kB)
Using cached django-6.1-py3-none-any.whl (8.4 MB)
Using cached asgiref-3.12.1-py3-none-any.whl (25 kB)
Using cached sqlparse-0.6.0-py3-none-any.whl (50 kB)
Using cached tzdata-2026.3-py2.py3-none-any.whl (348 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.12.1 django-6.1 sqlparse-0.6.0 tzdata-2026.3

[notice] A new release of pip is available: 25.3 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

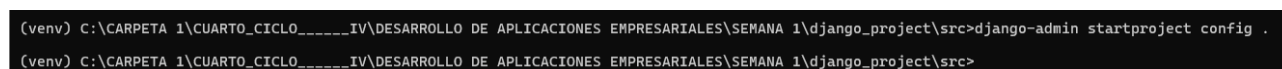
(venv) C:\CARPETA 1\CUARTO_CICLO_____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project\src>python -m django --version
6.1

(venv) C:\CARPETA 1\CUARTO_CICLO_____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project\src>
  
```

Se instaló Django dentro del entorno virtual activado usando pip install django, y se verificó la instalación con python -m django --version, confirmando la versión 6.1 instalada correctamente en el entorno del proyecto.

3. Crear el proyecto con configuración separada. Usando django-admin, crea un proyecto llamado config dentro de la carpeta src/, de forma que manage.py quede en src/ y la configuración (settings.py, urls.py, etc.) quede en src/config/.

Figura 3



```

(venv) C:\CARPETA 1\CUARTO_CICLO_____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project\src>django-admin startproject config .
(venv) C:\CARPETA 1\CUARTO_CICLO_____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project\src>
  
```

Se creó el proyecto config dentro de la carpeta src/ usando `django-admin startproject config` . (con el punto al final), de modo que el archivo `manage.py` quedó en `src/` y los archivos de configuración (`settings.py`, `urls.py`, `asgi.py`, `wsgi.py`) quedaron dentro de `src/config/`.

4. Crear y registrar la aplicación core. Crea la aplicación core con `manage.py startapp` y regístrala en la lista `INSTALLED_APPS` de `src/config/settings.py`.

Figura 4

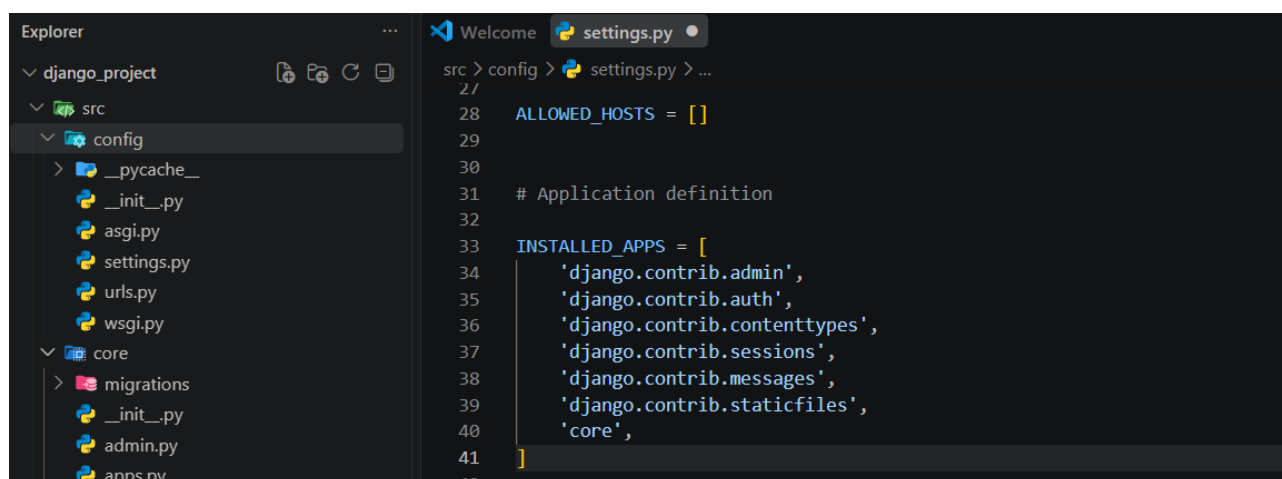
```
(venv) C:\CARPETA 1\CUARTO_CICLO_-----IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project\src>python manage.py startapp core

(venv) C:\CARPETA 1\CUARTO_CICLO_-----IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project\src>dir
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: 3EEF-14EC

Directorio de C:\CARPETA 1\CUARTO_CICLO_-----IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project\src
23/08/2026  14:06    <DIR>          .
23/08/2026  13:59    <DIR>          ..
23/08/2026  14:00    <DIR>          config
23/08/2026  14:06    <DIR>          core
23/08/2026  14:00             131,072 db.sqlite3
23/08/2026  13:56             684 manage.py
                2 archivos             131,756 bytes
                4 dirs  253,269,979,136 bytes libres

(venv) C:\CARPETA 1\CUARTO_CICLO_-----IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project\src>
```

Figura 5



```
src > config > settings.py > ...
27
28     ALLOWED_HOSTS = []
29
30
31     # Application definition
32
33     INSTALLED_APPS = [
34         'django.contrib.admin',
35         'django.contrib.auth',
36         'django.contrib.contenttypes',
37         'django.contrib.sessions',
38         'django.contrib.messages',
39         'django.contrib.staticfiles',
40         'core',
41     ]
42
```

Se creó la aplicación core con `python manage.py startapp core` y se registró en la lista `INSTALLED_APPS` del archivo `settings.py`, para que Django la reconozca como parte del proyecto.

5. Definir el modelo Item. En `src/core/models.py` define un modelo Item con los campos `name` (texto), `description` (texto largo, opcional) y `created_at` (fecha/hora de creación automática). Genera y aplica las migraciones correspondientes.

Figura 6

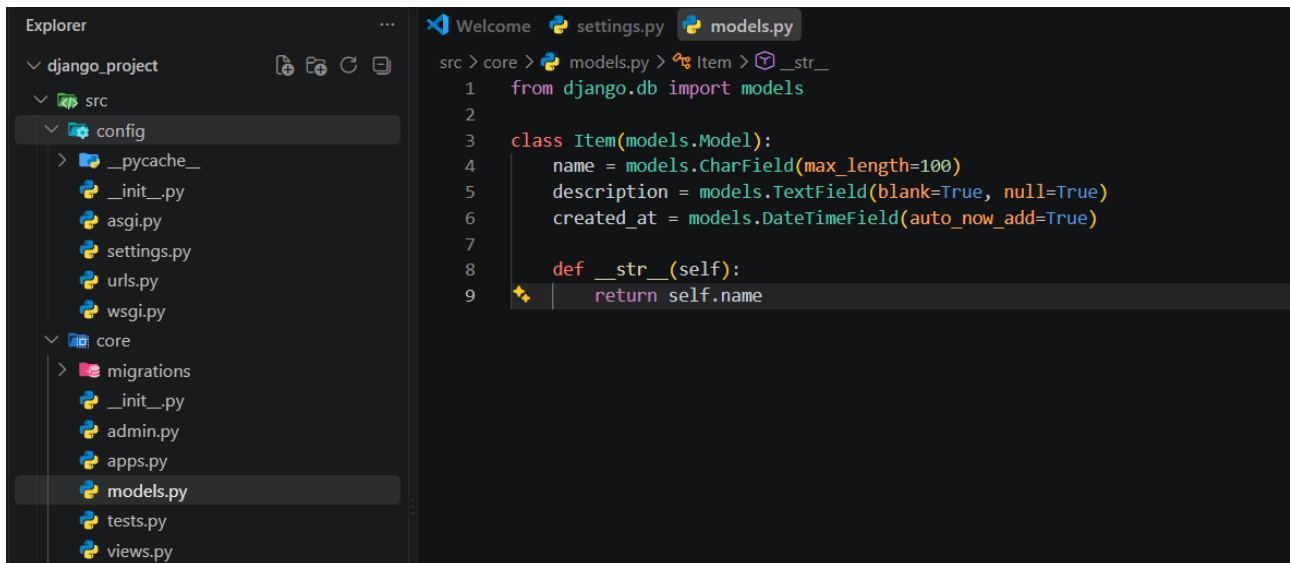
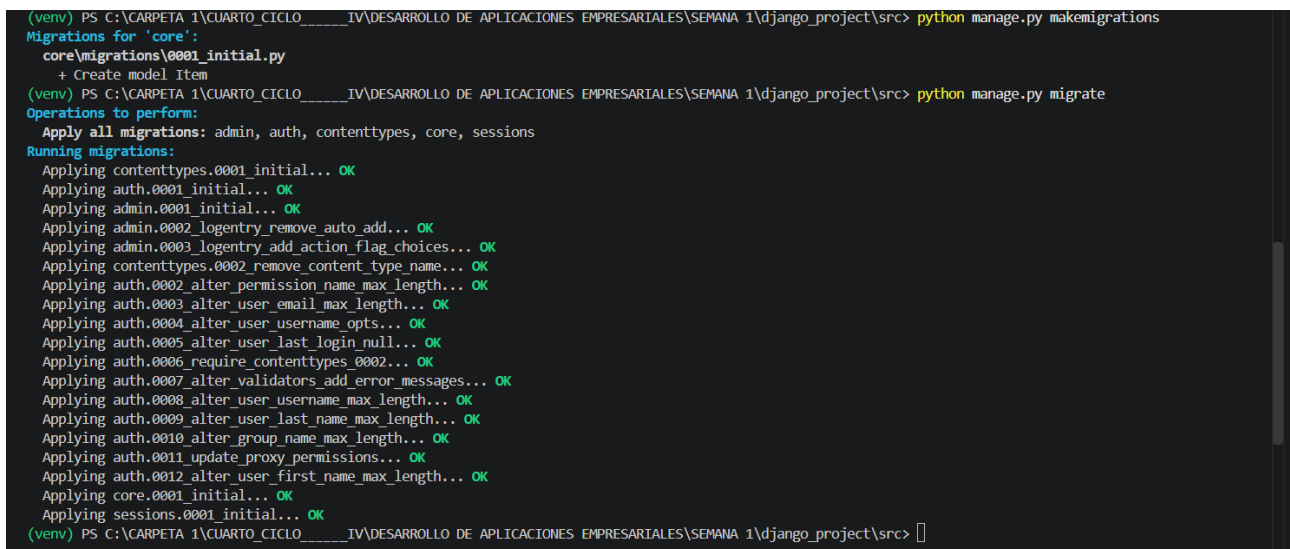


Figura 7



Se definió el modelo `Item` en `models.py` con los campos `name`, `description` y `created_at`. Luego se generaron y aplicaron las migraciones con `makemigrations` y `migrate`, creando la tabla correspondiente en la base de datos.

6. Crear la vista y las URLs. Implementa en `src/core/views.py` una vista `item_list` que obtenga todos los `Item` y los envíe a una plantilla. Configura las URLs de la app (`core/urls.py`) y enlázalas en `src/config/urls.py` mediante `include()`.

Figura 8

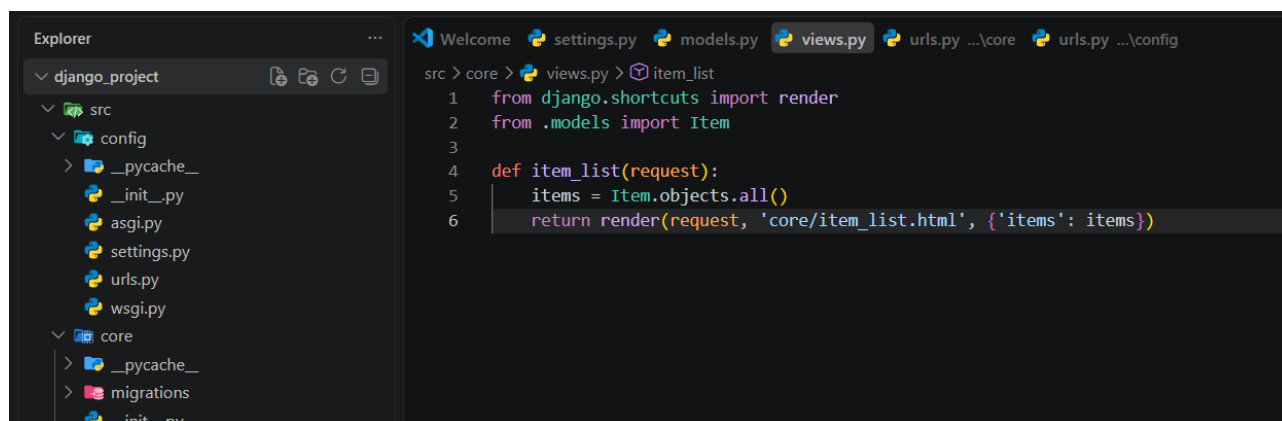


Figura 9

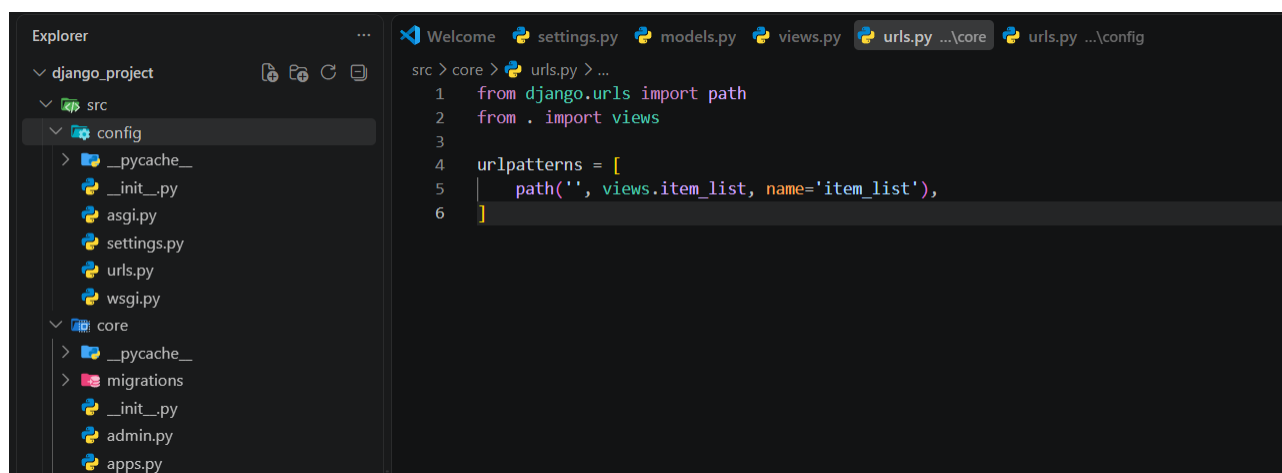
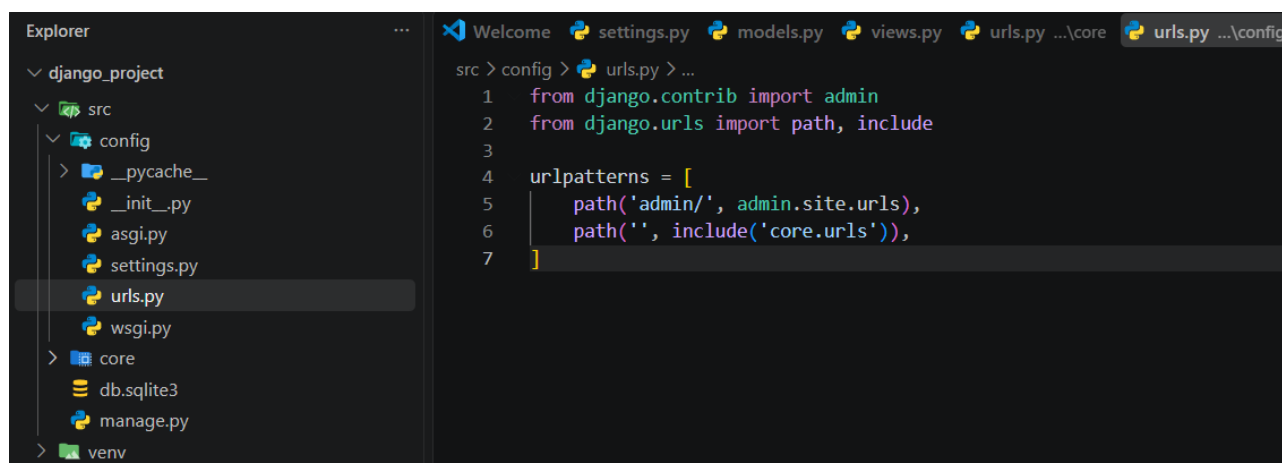


Figura 10



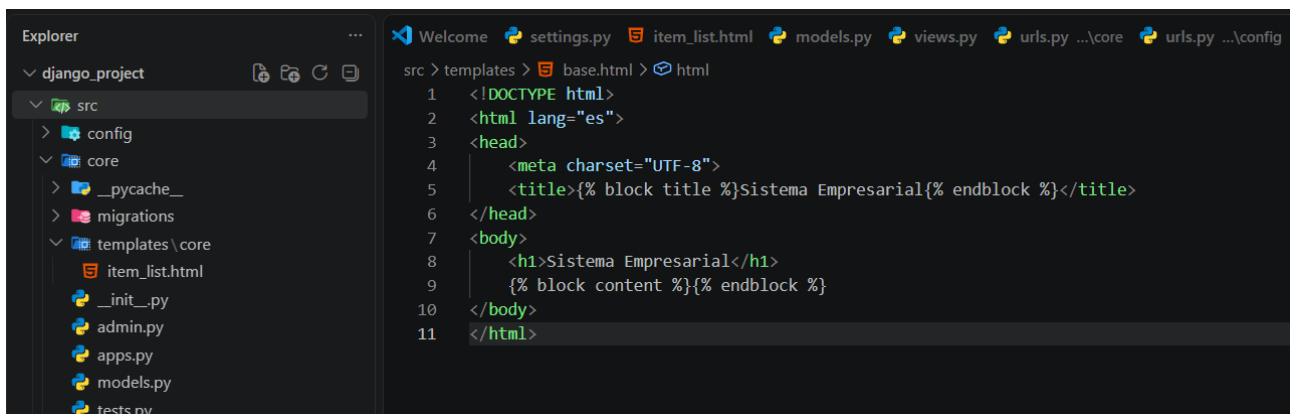
Se creó la vista `item_list` en `views.py`, que obtiene todos los registros de `Item` y los envía a una plantilla. Se configuraron las URLs de la app en `core/urls.py` y se enlazaron al proyecto principal mediante `include()` en `config/urls.py`.

7. Crear las plantillas. Crea una plantilla base (`base.html`) con una estructura HTML general, y una plantilla `core/item_list.html` que herede de la base y muestre el listado de ítems (usa `{% for %}` y `{% empty %}`).

Figura 11



Figura 12



Se creó la plantilla `base.html` con la estructura HTML general y bloques reutilizables, y la plantilla `item_list.html` que hereda de ella y muestra el listado de ítems usando `{% for %}` y `{% empty %}`.

- Configurar el administrador y cargar datos. Registra el modelo `Item` en `src/core/admin.py`, crea un superusuario, ingresa al panel de administración y registra al menos dos ítems de prueba.

Figura 13

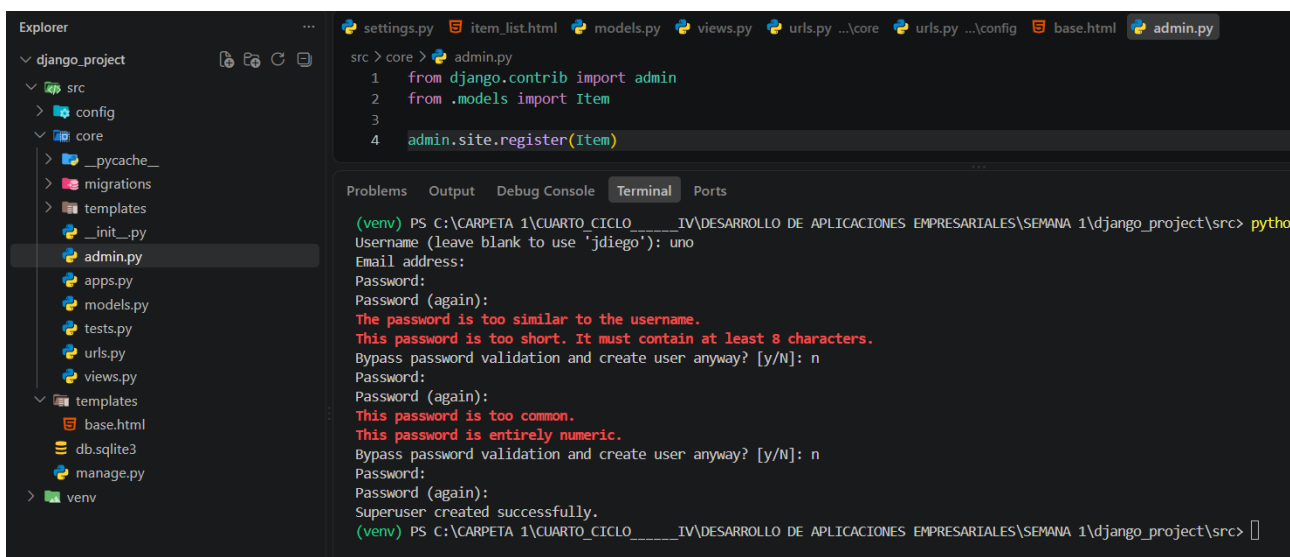


Figura 14

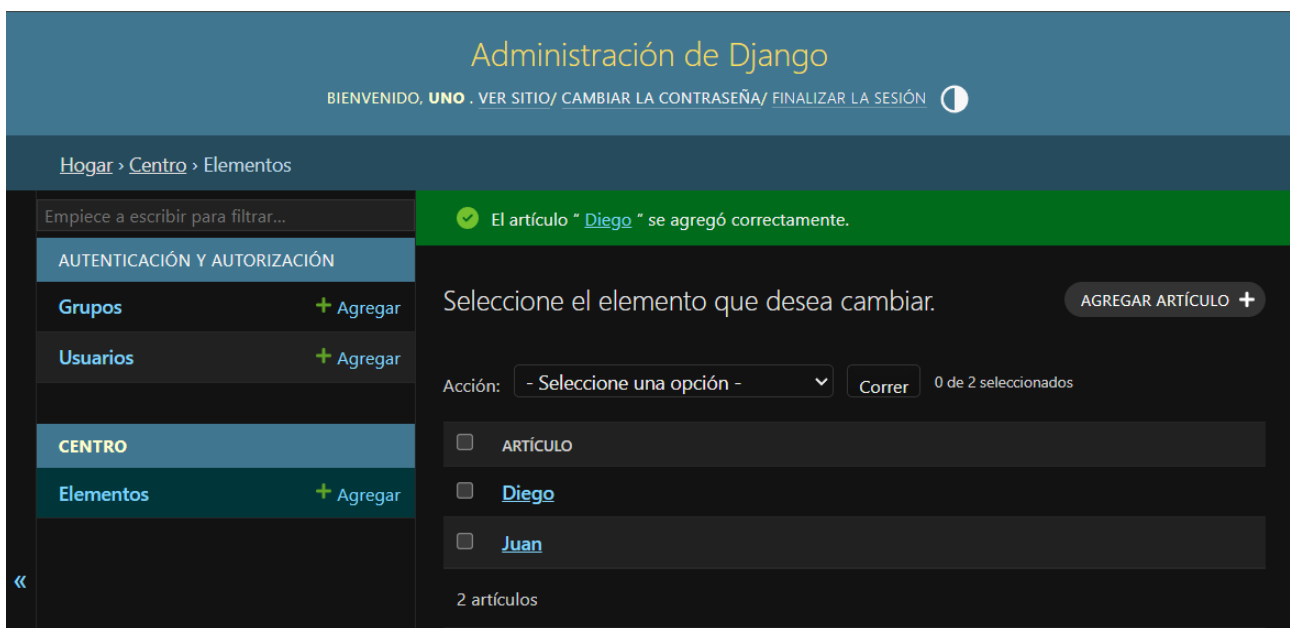

```
(venv) PS C:\CARPETA 1\CUARTO CICLO____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project\src> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
August 23, 2026 - 14:40:12
Django version 6.1, using settings 'config.settings'
Starting WSGI development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.1/howto/deployment/

[23/Aug/2026 14:40:19] "GET /admin/ HTTP/1.1" 302 0
[23/Aug/2026 14:40:19] "GET /admin/login/?next=/admin/ HTTP/1.1" 200 4307
[23/Aug/2026 14:40:20] "GET /static/admin/css/login.css HTTP/1.1" 200 951
[23/Aug/2026 14:40:20] "GET /static/admin/js/theme.js HTTP/1.1" 200 1708
[23/Aug/2026 14:40:20] "GET /static/admin/css/dark_mode.css HTTP/1.1" 200 3583
[23/Aug/2026 14:40:20] "GET /static/admin/css/base.css HTTP/1.1" 200 24538
[23/Aug/2026 14:40:20] "GET /static/admin/css/responsive.css HTTP/1.1" 200 15025
[23/Aug/2026 14:40:20] "GET /static/admin/css/nav_sidebar.css HTTP/1.1" 200 2814
[23/Aug/2026 14:40:20] "GET /static/admin/js/nav_sidebar.js HTTP/1.1" 200 3208
Not Found: /favicon.ico
[23/Aug/2026 14:40:20] "GET /favicon.ico HTTP/1.1" 404 2531
```

Figura 15



Se registró el modelo Item en admin.py para que sea gestionable desde el panel de administración, se creó un superusuario con createsuperuser y se cargaron dos ítems de prueba desde /admin/.

9. Verificar el funcionamiento. Levanta el servidor de desarrollo (runserver) y comprueba que la página principal (<http://127.0.0.1:8000/>) muestra el listado de ítems, y que el panel de administración (/admin/) funciona correctamente.

Figura 16

```
(venv) PS C:\CARPETA 1\CUARTO CICLO____IV\DESARROLLO DE APLICACIONES EMPRESARIALES\SEMANA 1\django_project\src> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
August 23, 2026 - 14:45:45
Django version 6.1, using settings 'config.settings'
Starting WSGI development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

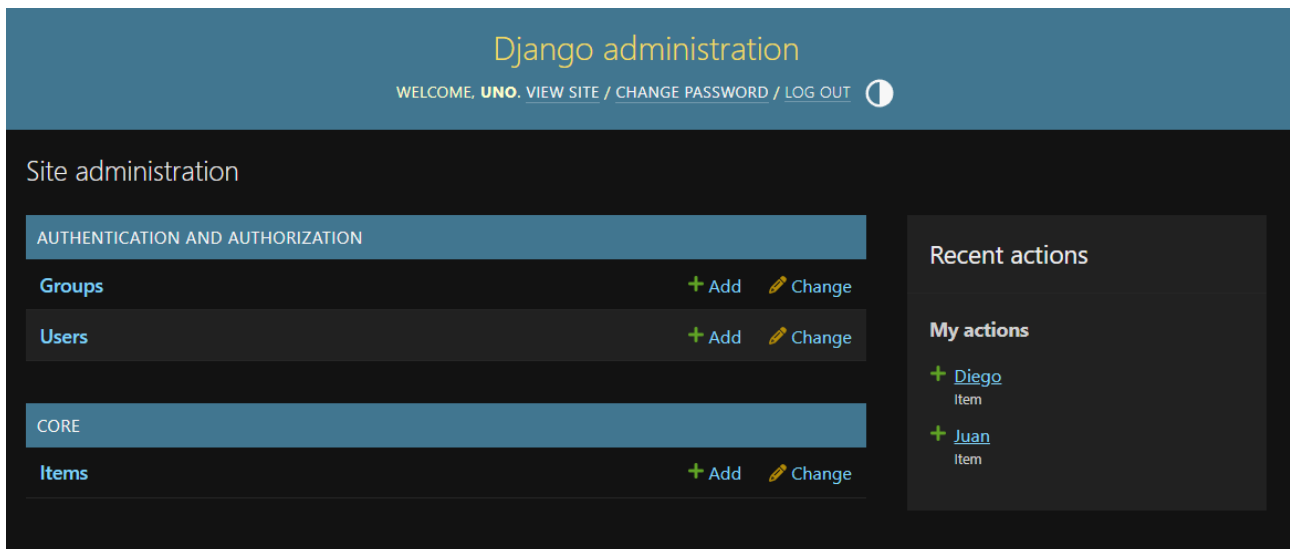
WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.1/howto/deployment/
```

Figura 17

Sistema Empresarial

- Juan — Hola este es mi primer nombre
- Diego — Hola este es mi segundo nombre

Figura 18



Se levantó el servidor de desarrollo con `python manage.py runserver` y se verificó que la página principal (`http://127.0.0.1:8000/`) muestra correctamente el listado de ítems cargados previamente, y que el panel de administración (`/admin/`) permite iniciar sesión y gestionar los registros del modelo `Item`.

10. Documentar y subir el proyecto. Genera el archivo `requirements.txt` con las dependencias del proyecto, crea un `README.md` que explique la estructura y el modo de instalación/ejecución, y sube el proyecto a un repositorio de GitHub.

Figura 19

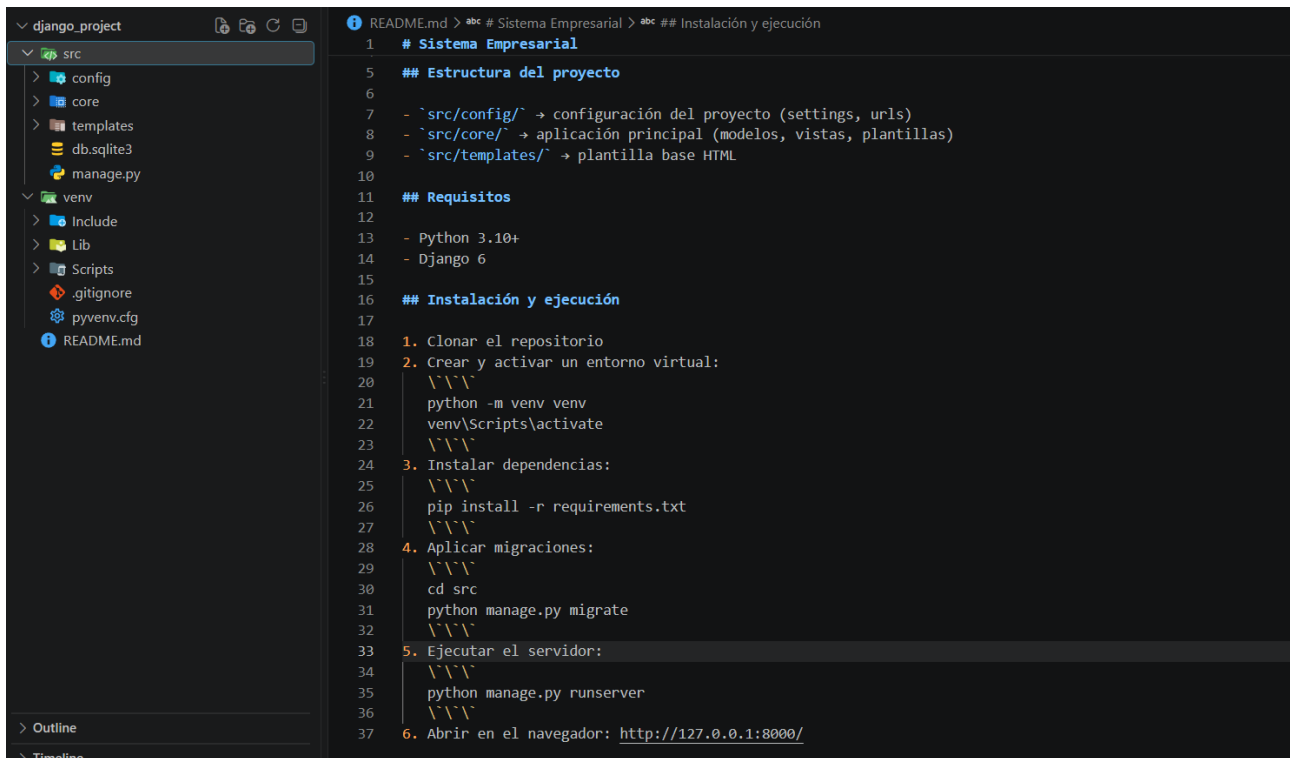
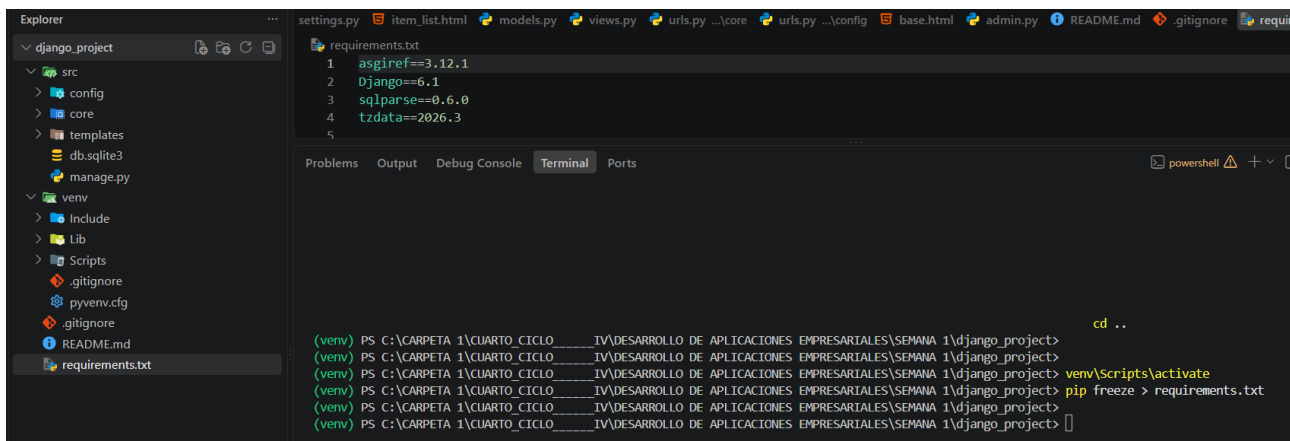


Figura 20



Se generó el archivo requirements.txt con pip freeze para documentar las dependencias del proyecto, se creó un README.md explicando la estructura, requisitos e instrucciones de instalación y ejecución, y finalmente se inicializó un repositorio Git y se subió el proyecto completo a GitHub.

Reto (opcional)

- Agrega estilos CSS a la plantilla base para mejorar la presentación del listado de ítems.
- Agrega un elemento de interactividad en el frontend (por ejemplo, un filtro o buscador con JavaScript).
- Con apoyo de IA, crea uno o más endpoints (API) para el modelo Item y consúmelos desde un frontend construido con HTML, CSS y JavaScript vanilla, o con React.

Figura 21

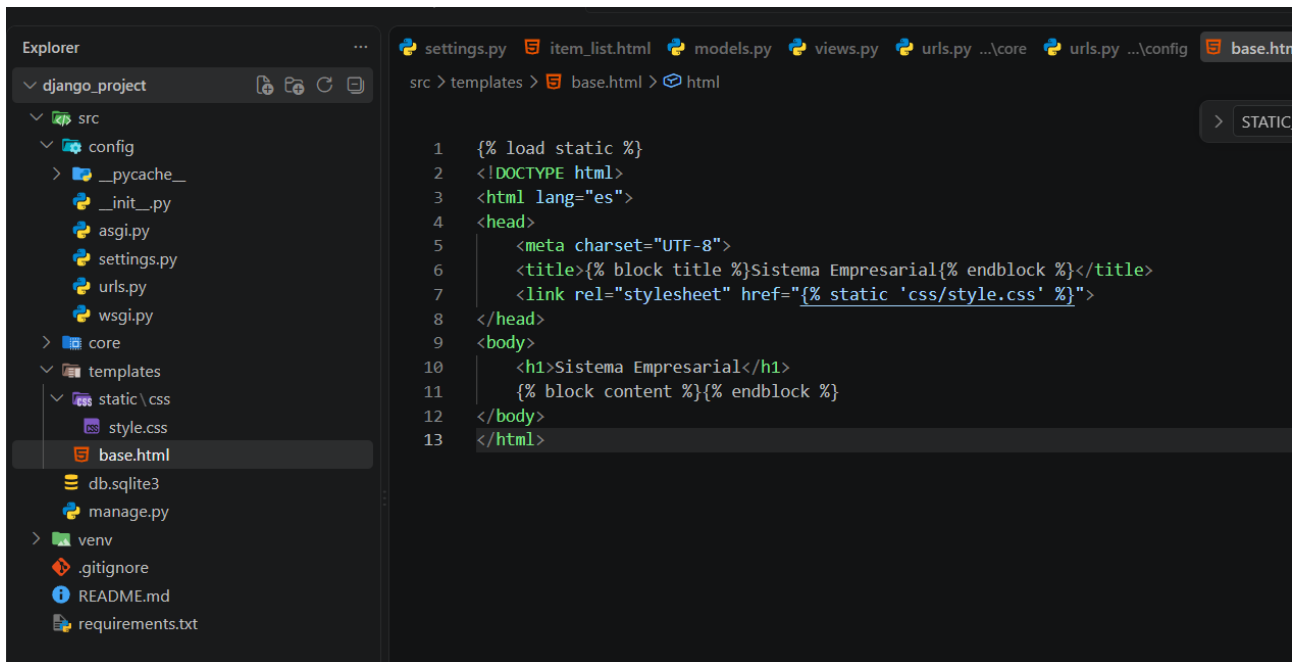


Figura 22

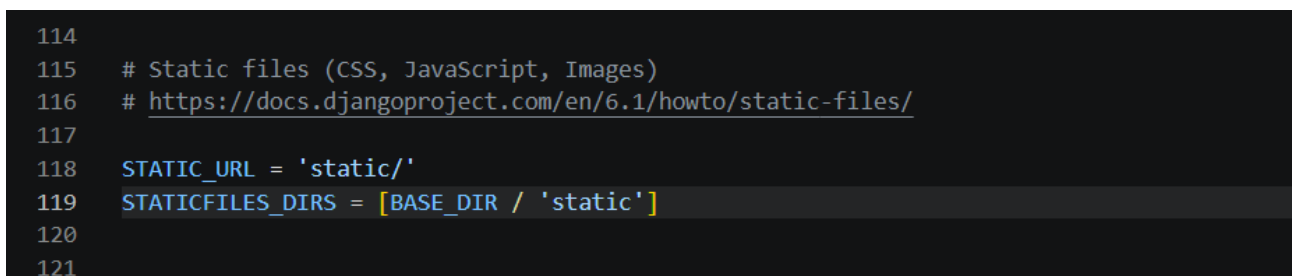


Figura 23

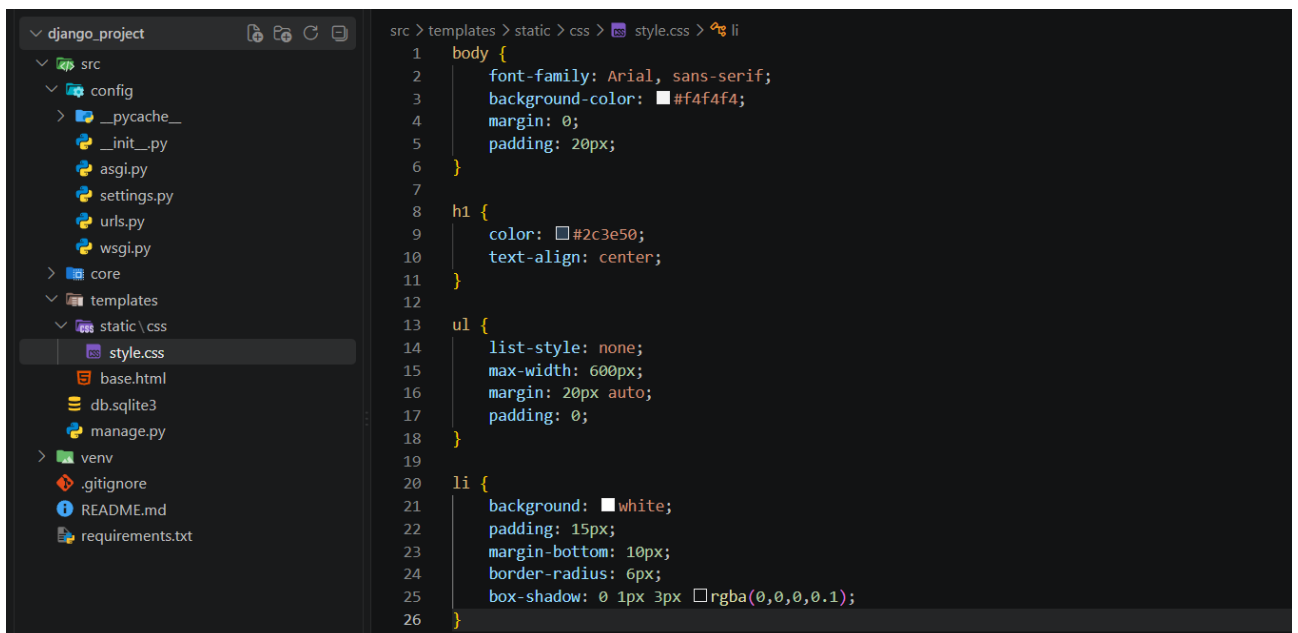


Figura 24

Sistema Empresarial

Juan — Hola este es mi primer nombre

Diego — Hola este es mi segundo nombre

Entregables

- Informe con esta misma plantilla del laboratorio: capturas de pantalla de cada ejercicio, con una breve explicación de lo realizado.
- Repositorio: enlace al repositorio de GitHub con el proyecto completo.
- Sustentación: el estudiante debe estar en capacidad de explicar y ejecutar su proyecto en clase.
- Subir al canvas en formato pdf

<https://github.com/juandiegoramosenriquez-sys/sistema-empresarial-django>

Solución de los ejercicios

A continuación se presenta una posible solución de referencia para el docente, con los comandos y el código esperado en cada ejercicio.

1-2. Preparar el entorno e instalar Django

```
mkdir django_project && cd django_project
python3 -m venv venv
source venv/bin/activate          # En Windows: venv\Scripts\activate
mkdir src && cd src
pip3 install django
python3 -m django --version
```

3. Crear el proyecto config

```
# Estando dentro de src/
django-admin startproject config .
```

Esto genera manage.py en src/ y la carpeta config/ con settings.py, urls.py, asgi.py y wsgi.py.

4. Crear y registrar la app core

```
python3 manage.py startapp core
```

En src/config/settings.py:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
```

```
'django.contrib.sessions',  
'django.contrib.messages',  
'django.contrib.staticfiles',  
'core', # Nuestra aplicación  
)
```

Conclusiones:

- Se construyó la primera versión de un sistema empresarial en Django, comprendiendo su estructura básica (proyecto, app, modelos, vistas, plantillas y administrador) bajo el patrón MVT.
- Se reforzó el uso de entornos virtuales, migraciones y herencia de plantillas para el desarrollo ordenado de la aplicación.
- Se aplicaron buenas prácticas de control de versiones con Git y GitHub, documentando el proyecto con README.md y .gitignore, sentando las bases para futuros desarrollos.